

Documentación técnica · v1.0

SmartCommerce

Documentación técnica del proyecto: arquitectura, modelo de datos, API, seguridad y guía de incorporación para desarrolladores.

- Producto:** Plataforma de e-commerce (backend + frontend)
- Stack:** Java 21 · Spring Boot 4 · PostgreSQL 15 · Spring Security (JWT) · React (Vite) · Tailwind CSS · Docker
- Repositorio:** monorepo (backend Spring Boot + frontend React)
- Autor:** Darius Adrian Benta
- Estado:** en desarrollo activo · Documento dirigido a desarrolladores que se incorporan al proyecto.

Índice

1. Introducción

1. Propósito de este documento
2. Qué es SmartCommerce
3. A quién va dirigido

2. Arquitectura

1. Vista de alto nivel
2. Monolito modular y módulos
3. Arquitectura por capas
4. Ciclo de vida de una petición
5. Decisiones de diseño

3. Stack tecnológico

4. Estructura del proyecto

5. Modelo de datos

1. Motor y conexión
2. Diagrama entidad-relación
3. Tablas
4. Enumerados
5. Reglas de integridad

6. Seguridad

1. Modelo de seguridad
2. Flujo de autenticación
3. Autorización (rutas y roles)
4. Componentes

7. Referencia de la API REST

1. Convenciones
2. Autenticación
3. Usuarios
4. Productos
5. Categorías
6. Carrito
7. Lista de deseos
8. Direcciones

8. Módulos del backend

9. Frontend

10. Configuración

11. Puesta en marcha (onboarding)

12. Datos de demostración (seeders)

13. Manejo de errores

14. Convenciones de desarrollo y flujo Git

15. Roadmap

16. Glosario

1. Introducción

1.1 Propósito de este documento

Esta documentación describe la arquitectura, el modelo de datos, la API y el funcionamiento interno de **SmartCommerce**. Su objetivo es que **cualquier desarrollador que se incorpore al proyecto pueda entenderlo y ser productivo sin necesidad de explicaciones adicionales**: cómo está estructurado, por qué se tomaron ciertas decisiones, cómo arrancarlo y cómo extenderlo siguiendo las convenciones existentes.

1.2 Qué es SmartCommerce

SmartCommerce es una **aplicación de comercio electrónico** tipo marketplace. Consta de:

- Un **backend** en Spring Boot que expone una API REST, aplica las reglas de negocio y persiste los datos en PostgreSQL.
- Un **frontend** en React (SPA) que consume esa API y ofrece la interfaz de usuario.
- Una **base de datos** PostgreSQL ejecutada en un contenedor Docker.

Funcionalmente cubre: registro e inicio de sesión con roles, catálogo de productos con búsqueda/filtros/paginación, categorías jerárquicas, carrito de la compra, lista de deseos (favoritos) y gestión de direcciones de usuario.

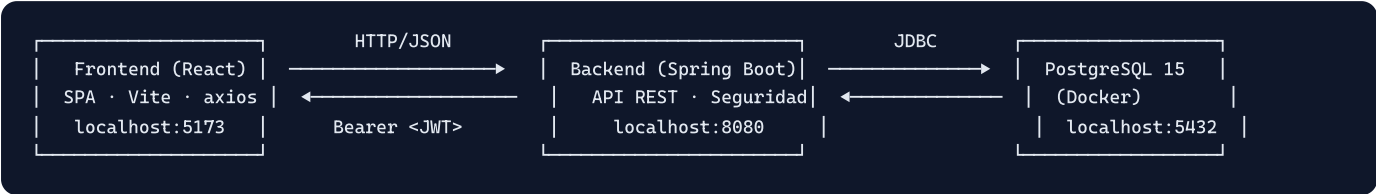
1.3 A quién va dirigido

A desarrolladores backend, frontend o full-stack que trabajen en el proyecto. Se asume conocimiento general de Java, HTTP/REST y JavaScript, pero se explica el *porqué* de cada decisión para facilitar la incorporación.

2. Arquitectura

2.1 Vista de alto nivel

Tres piezas desacopladas que se comunican por HTTP:



El frontend nunca accede a la base de datos directamente: todo pasa por la API. La autenticación viaja como un token JWT en la cabecera `Authorization`.

2.2 Monolito modular y módulos

El backend es un **monolito modular**: una única aplicación desplegable, organizada internamente en módulos por dominio de negocio. Cada módulo es autónomo y contiene todas sus capas.

Módulo	Responsabilidad
auth	Registro e inicio de sesión; emisión del token JWT.
user	Perfil del usuario y administración de usuarios (listar, consultar, desactivar, promover a admin).
product	Catálogo: CRUD de administrador, listado público con filtros y paginación, detalle.
productimage	Imágenes asociadas a productos.
category	Categorías jerárquicas (con subcategorías y <code>slug</code>).
cart	Carrito de la compra: añadir, actualizar cantidad, eliminar, vaciar; validación de stock.
wishlist	Lista de deseos (favoritos) por usuario.
address	Direcciones del usuario y dirección predeterminada.
security	Configuración de seguridad, filtro JWT y servicio de tokens (transversal).
config	Inicializadores de datos (admin y catálogo de demo).
exception	Manejo global de excepciones y respuesta de error unificada.
common	Utilidades compartidas (p. ej. generación de <code>slug</code>).

2.3 Arquitectura por capas

Dentro de cada módulo, el código se organiza en capas con una responsabilidad única. Los datos fluyen de arriba a abajo, y las entidades se convierten a DTOs antes de salir:

Controller → recibe la petición HTTP, valida la entrada, delega. (Sin lógica de negocio.)

|

Service → reglas de negocio (stock, permisos, cálculos). Interfaz + implementación (...ServiceImpl).

|

Repository → acceso a datos con Spring Data JPA.

|

Entity → representación de la tabla (solo dentro del backend).

DTO → objeto de entrada/salida de la API (nunca se exponen entidades).

Mapper → conversión Entity ↔ DTO con MapStruct.

2.4 Ciclo de vida de una petición

Ejemplo: actualizar la cantidad de una línea del carrito.

```
1. Frontend → PUT /api/v1/cart/items/5 { "quantity": 3 } (+ Authorization: Bearer <token>)
2. JwtAuthFilter → valida el token, carga el usuario y lo coloca en el SecurityContext.
3. SecurityConfig → comprueba que la ruta requiere autenticación (cumplida).
4. CartController → extrae el usuario y el productId; llama al servicio.
5. CartServiceImpl → valida stock, actualiza la cantidad (@Transactional).
6. CartItemRepo → persiste el cambio en PostgreSQL.
7. CartMapper → convierte la entidad Cart a CartResponseDTO (con subtotales y total).
8. Respuesta → 200 OK con el carrito actualizado en JSON.
```

2.5 Decisiones de diseño

Decisión	Motivo	Alternativa descartada
Monolito modular	El dominio es un único e-commerce; da orden y velocidad sin la complejidad operativa de los microservicios. Los módulos quedan desacoplados y extraíbles a futuro.	Microservicios (sobredimensionado).
Arquitectura por capas	Separa responsabilidades; facilita localizar, testear y mantener.	Lógica en los controladores.
PostgreSQL (relacional)	Dominio fuertemente relacional con necesidad de integridad referencial y transacciones.	NoSQL.
JWT stateless	API sin estado, escalable y desacoplada de una SPA.	Sesiones en servidor.
DTOs + MapStruct	Protege el modelo interno, define un contrato de API estable y evita <i>boilerplate</i> de mapeo.	Exponer entidades / mapeo manual.
Frontend React desacoplado	Permite desarrollar, desplegar y escalar front y back por separado.	Renderizado en servidor.

3. Stack tecnológico

Capa	Tecnología	Para qué
Backend	Java 21	Lenguaje base (LTS).
	Spring Boot 4	Framework de la aplicación (Web MVC, autoconfiguración, servidor embebido).
	Spring Data JPA / Hibernate	Persistencia (ORM) sobre PostgreSQL.
	Spring Security + jjwt	Autenticación y autorización con JWT.
	Jakarta Validation	Validación declarativa de las peticiones.
	MapStruct	Mapeo Entity ↔ DTO en tiempo de compilación.
	Lombok	Reducción de código repetitivo (getters, constructores, builder...).
	springdoc-openapi (Swagger UI)	Documentación interactiva de la API.
Frontend	React + Vite	Interfaz SPA y servidor de desarrollo con recarga en caliente.
	React Router	Navegación sin recarga.
	axios	Cliente HTTP con interceptores (JWT y manejo de 401).
	Tailwind CSS	Estilos mediante clases utilitarias.
	Context API + Hooks	Estado global (sesión, favoritos) y lógica reutilizable.
Base de datos	PostgreSQL 15	Almacenamiento relacional.
Infraestructura	Docker / Docker Compose	Ejecución reproducible de la base de datos (y, a futuro, de toda la app).
Build	Maven (wrapper) · npm	Gestión de dependencias y compilación del backend y del frontend.

4. Estructura del proyecto

El repositorio es un **monorepo**: backend y frontend conviven para facilitar su orquestación conjunta.

```
smartcommerce/                                # raíz del repositorio (backend)
├── src/main/java/com/smartcommerce/
│   ├── SmartcommerceApplication.java          # punto de entrada (@SpringBootApplication)
│   ├── auth/                                  # registro / login / JWT
│   ├── user/                                  # perfil y administración de usuarios
│   ├── product/                              # catálogo (controller, service, repo, dto, mapper, entity, spec)
│   ├── productimage/                         # imágenes de producto
│   ├── category/                             # categorías jerárquicas
│   ├── cart/                                 # carrito y líneas de carrito
│   ├── wishlist/                             # lista de deseos
│   ├── address/                              # direcciones de usuario
│   ├── security/                             # SecurityConfig · JwtAuthFilter · JwtService
│   ├── config/                               # DataInitializer · CatalogInitializer (seeders)
│   ├── common/                               # utilidades (SlugUtils)
│   └── exception/                            # GlobalExceptionHandler · ErrorResponse · excepciones
├── # Patrón por módulo: controller · service (+impl) · repository · dto/{request,response} · mapper · entity
├── src/main/resources/application.properties
├── src/test/                                  # pruebas
├── frontend/                                 # cliente React (Vite)
│   └── src/
│       ├── api/                              # axiosClient (instancia + interceptores)
│       ├── services/                         # llamadas a la API por dominio (productService, cartService...)
│       ├── context/                          # estado global (AuthContext, WishlistContext)
│       ├── hooks/                            # hooks reutilizables (useDebounce, useAuth, useWishlist)
│       ├── components/                       # componentes de UI (ProductCard, PriceRangeSlider...)
│       ├── pages/                           # vistas (Home, Cart, Favorites, Login...)
│       ├── layouts/                          # estructura común (MainLayout, Sidebar)
│       └── routes/                           # AppRoutes, ProtectedRoute
├── docs/                                     # documentación
├── docker-compose.yml                       # orquestación de la base de datos
└── pom.xml                                  # dependencias y build del backend
```

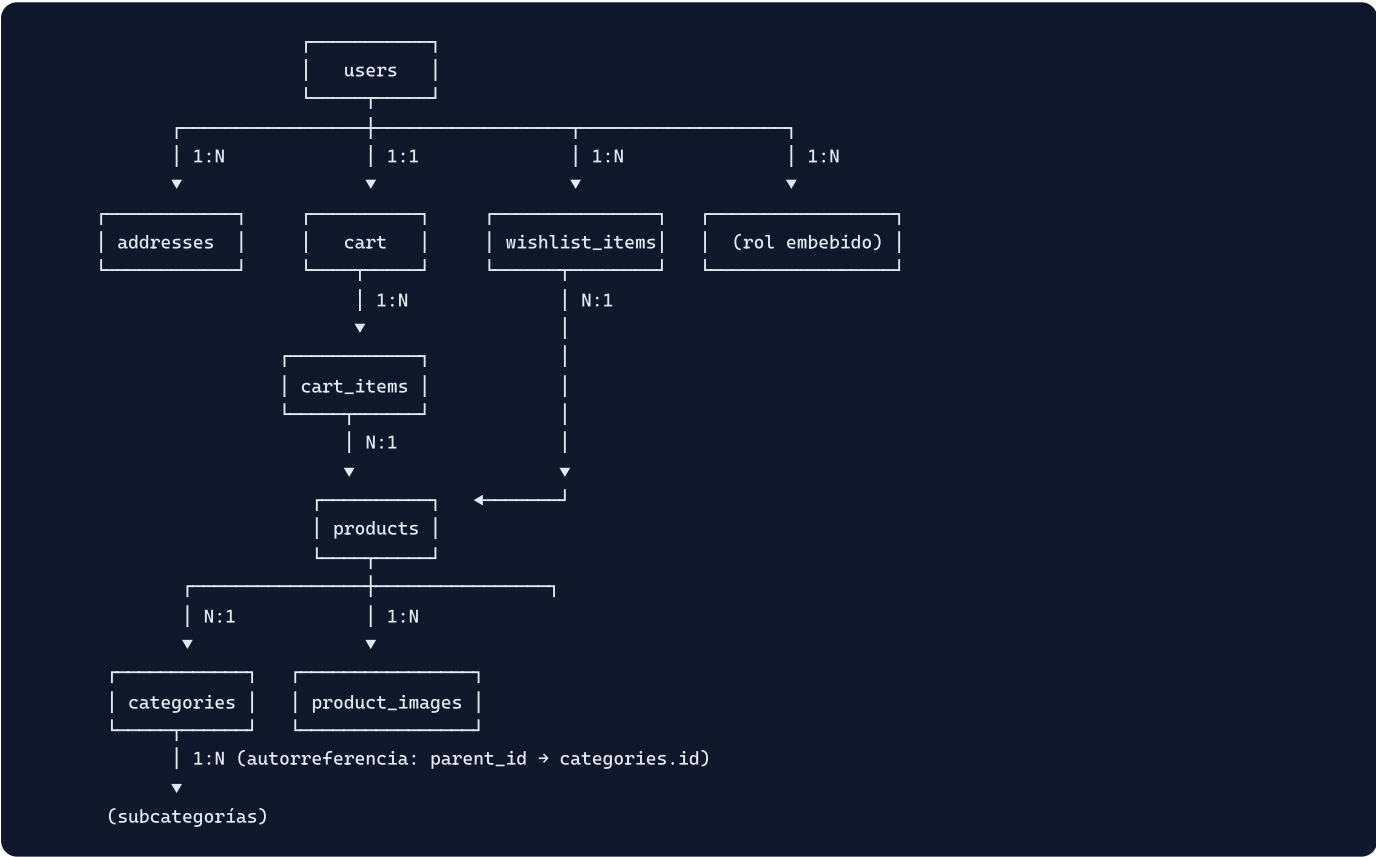
5. Modelo de datos

5.1 Motor y conexión

El motor es **PostgreSQL 15**. El esquema se genera automáticamente a partir de las entidades JPA (`spring.jpa.hibernate.ddl-auto=update`).

Parámetro	Valor por defecto
Host / Puerto	localhost:5432
Base de datos	smartcommerce
Usuario / Contraseña	postgres / postgres
URL JDBC	jdbc:postgresql://localhost:5432/smartcommerce

5.2 Diagrama entidad-relación



5.3 Tablas

users

Usuarios de la aplicación. La entidad implementa `UserDetails` de Spring Security.

Columna	Tipo	Restricciones
id	bigint	PK, autogenerado
first_name / last_name	varchar	NOT NULL
email	varchar	NOT NULL, UNIQUE (es el "username")
password	varchar	NOT NULL (cifrada con BCrypt)
phone	varchar	opcional
birth_date	date	opcional
role	varchar (enum)	NOT NULL · <code>USER</code> <code>ADMIN</code>
active	boolean	NOT NULL (por defecto true)
created_at	timestamp	NOT NULL, no modificable (se fija en <code>@PrePersist</code>)

products

Columna	Tipo	Restricciones
id	bigint	PK, autogenerado
name	varchar	NOT NULL
description	varchar	opcional
price	numeric	NOT NULL (BigDecimal)
stock	integer	NOT NULL
status	varchar (enum)	NOT NULL · <code>ACTIVE</code> <code>OUT_OF_STOCK</code> <code>DISABLED</code>
brand	varchar	opcional
created_at	timestamp	se fija en <code>@PrePersist</code>
category_id	bigint	FK → categories.id

product_images

Columna	Tipo	Restricciones
id	bigint	PK
image_url	varchar	NOT NULL
product_id	bigint	FK → products.id, NOT NULL

categories

Categorías jerárquicas: una categoría puede tener una categoría padre (autorreferencia).

Columna	Tipo	Restricciones
id	bigint	PK
name	varchar	NOT NULL
slug	varchar	NOT NULL, UNIQUE (URL-friendly, se genera del nombre)
active	boolean	NOT NULL
parent_id	bigint	FK → categories.id (nulo si es raíz)

cart / cart_items

Tabla	Columna	Restricciones
cart	id	PK
cart	user_id	FK → users.id, NOT NULL, UNIQUE (1 carrito por usuario)
cart_items	id	PK
cart_items	cart_id	FK → cart.id, NOT NULL
cart_items	product_id	FK → products.id, NOT NULL
cart_items	quantity	NOT NULL

La colección de líneas del carrito usa `@OrderBy("id ASC")` para devolver los items en orden estable (evita que la interfaz reordene las filas). `cascade=ALL + orphanRemoval=true` : al eliminar una línea de la colección, se borra de la base de datos.

wishlist_items

Columna	Restricciones
id	PK
user_id	FK → users.id, NOT NULL
product_id	FK → products.id, NOT NULL
(user_id, product_id)	UNIQUE (un producto no se puede añadir dos veces a la misma lista)

addresses

Columna	Restricciones
id	PK
user_id	FK → users.id, NOT NULL
street	NOT NULL
address_line2	opcional
city / province / postal_code / country	NOT NULL
is_default	NOT NULL (marca la dirección predeterminada)

5.4 Enumerados

Enum	Valores	Uso
Role	USER , ADMIN	Rol del usuario; controla el acceso.
ProductStatus	ACTIVE , OUT_OF_STOCK , DISABLED	Estado del producto; se recalcula según el stock (0 → OUT_OF_STOCK) y al "borrar" (DISABLED).

5.5 Reglas de integridad

- Las claves foráneas (product_id , user_id ...) garantizan que no existan referencias huérfanas: no se puede borrar un producto que esté referenciado por una línea de carrito o de wishlist.
- email (users) y slug (categories) son únicos.
- La pareja (user_id, product_id) en wishlist_items es única.
- El "borrado" de un producto es lógico (pasa a DISABLED), no físico.

6. Seguridad

6.1 Modelo de seguridad

Autenticación **stateless** basada en **JWT**. El servidor no guarda sesiones: cada petición se autentica de forma independiente validando el token. La autorización se basa en **roles** (**USER** , **ADMIN**).

6.2 Flujo de autenticación

1. POST /api/v1/auth/login { email, password }
2. El backend verifica la contraseña (BCrypt) y genera un JWT firmado (HMAC-SHA).
3. Devuelve { token, expiresIn, userId, email, role }.
4. El cliente guarda el token y lo envía en cada petición: Authorization: Bearer <token>
5. JwtAuthFilter valida firma y caducidad, carga el usuario y lo pone en el SecurityContext.
6. El controlador ya conoce al usuario autenticado.

El token caduca en **1 hora** (`jwt.expiration=3600000` ms). Pasado ese tiempo hay que volver a autenticarse.

6.3 Autorización (rutas y roles)

Patrón de ruta	Acceso
/api/v1/auth/**	Público (registro y login).
GET /api/v1/products/**	Público (ver catálogo y detalle).
/swagger-ui/** , /v3/api-docs/**	Público (documentación).
/api/v1/admin/**	Solo rol ADMIN .
Endpoints admin de categorías (/categories/admin/**)	Solo ADMIN (vía <code>@PreAuthorize</code> a nivel de método).
Cualquier otra ruta	Requiere estar autenticado.

Respuestas de seguridad: **401 Unauthorized** = falta el token o es inválido; **403 Forbidden** = autenticado pero sin permiso (p. ej. un **USER** intentando una acción de **ADMIN**).

6.4 Componentes

Clase	Responsabilidad
<code>SecurityConfig</code>	Define la cadena de filtros: desactiva CSRF, activa CORS, política STATELESS, reglas de autorización por ruta, y registra el filtro JWT. Declara los beans <code>PasswordEncoder</code> (BCrypt) y <code>AuthenticationManager</code> .
<code>JwtAuthFilter</code>	Filtro que se ejecuta una vez por petición: extrae el token de la cabecera, lo valida y establece la autenticación en el contexto.
<code>JwtService</code>	Genera y valida tokens (firma HMAC con la clave secreta, extrae el email, comprueba la caducidad).
<code>UserDetailsServiceImpl</code>	Carga el usuario por email para que Spring Security lo autentique.

CORS: el backend autoriza explícitamente el origen del frontend (`http://localhost:5173`) y los métodos GET/POST/PUT/PATCH/DELETE, porque frontend y backend corren en orígenes distintos.

7. Referencia de la API REST

7.1 Convenciones

- **URL base:** `http://localhost:8080/api/v1`
- **Formato:** JSON en peticiones y respuestas.
- **Autenticación:** cabecera `Authorization: Bearer <token>` en los endpoints protegidos.
- **Paginación** (listado de productos): parámetros `page` (0-based), `size` y `sort` (p. ej. `sort=price,asc`). La respuesta sigue el formato `Page` de Spring: `{ content, totalElements, totalPages, number, size }`.
- **Documentación interactiva:** Swagger UI en `/swagger-ui.html`.

Leyenda de acceso: público sin token · autenticado requiere login · ADMIN solo administradores.

7.2 Autenticación · `/api/v1/auth`

POST `/auth/register` público

Registra un usuario y devuelve un token. Cuerpo: `{ firstName, lastName, phone, birthDate (pasado), email, password (mín. 8) }`. Respuesta `AuthResponse`: `{ token, expiresIn, userId, email, role }`.

POST `/auth/login` público

Inicia sesión. Cuerpo: `{ email, password }`. Respuesta: `AuthResponse`. Error `401` si las credenciales son incorrectas.

7.3 Usuarios · `/api/v1`

GET `/users/me` autenticado

Devuelve el perfil del usuario autenticado (`UserResponseDTO` : id, nombre, apellidos, teléfono, fecha de nacimiento, email, rol, activo, fecha de alta).

PUT `/users/me` autenticado

Actualiza el perfil propio.

GET `/admin/users` ADMIN

Lista todos los usuarios.

GET `/admin/users/{userId}` ADMIN

Consulta un usuario por id.

PATCH `/admin/users/{userId}/deactivate` ADMIN

Desactiva un usuario.

PATCH `/admin/users/{userId}/promote` ADMIN

Promueve un usuario a rol `ADMIN`.

7.4 Productos · `/api/v1`

GET `/products` público

Listado paginado con filtros por query: `name`, `categoryId`, `minPrice`, `maxPrice`, `brand`, más `page` / `size` / `sort`. Cada producto (`ProductResponseDTO`): `{ id, name, description, price, stock, status, brand, categoryId, categoryName, imageUrls }`.

GET /products/{id}

público

Detalle de un producto. `404` si no existe.**POST** /admin/products

ADMIN

Crea un producto. Cuerpo: `{ name, description?, price (>0), stock (≥0), brand?, categoryId, imageUrls? }`. Respuesta `201`.**PUT** /admin/products/{id}

ADMIN

Actualiza un producto (campos parciales).

PUT /admin/products/{id}/images

ADMIN

Reemplaza las imágenes del producto.

DELETE /admin/products/{id}

ADMIN

Borrado lógico: el producto pasa a estado `DISABLED`.

7.5 Categorías · /api/v1/categories

GET /categories

autenticado

Categorías raíz activas. `CategoryResponseDTO`: `{ id, name, slug, parentId, parentName }`.**GET** /categories/{id}

autenticado

Categoría por id.

GET /categories/{id}/children

autenticado

Subcategorías activas de una categoría.

GET /categories/admin

ADMIN

Todas las categorías (incluidas inactivas).

POST /categories/admin

ADMIN

Crea una categoría. Cuerpo: `{ name, parentId? }`. El `slug` y `active` se generan en el backend.**PUT** /categories/admin/{id}

ADMIN

Actualiza una categoría (con validación de ciclos en la jerarquía).

DELETE /categories/admin/{id}

ADMIN

Desactiva una categoría (no permite borrar si tiene subcategorías activas).

7.6 Carrito · /api/v1/cart

Todos requieren autenticación; operan sobre el carrito del usuario autenticado y devuelven el carrito completo (`CartResponseDTO`: `{ id, items: [{ productId, productName, productPrice, productImage, quantity, subtotal }], total }`).**GET** /cart

autenticado

Devuelve (o crea si no existe) el carrito del usuario.

POST /cart/items

autenticado

Añade un producto. Cuerpo: `{ productId, quantity }`. Valida stock y estado del producto. Respuesta `201`.

PUT /cart/items/{productId}

autenticado

Fija la cantidad de una línea. Cuerpo: `{ quantity }`.

DELETE /cart/items/{productId}

autenticado

Elimina una línea del carrito.

DELETE /cart

autenticado

Vacía el carrito. Respuesta `204`.

7.7 Lista de deseos · `/api/v1/wishlist`

GET /wishlist

autenticado

Productos favoritos del usuario.

POST /wishlist/items/{productId}

autenticado

Añade un producto a favoritos (idempotente por la restricción única usuario+producto).

DELETE /wishlist/items/{productId}

autenticado

Quita un producto de favoritos.

7.8 Direcciones · `/api/v1/addresses`

GET /addresses

autenticado

Direcciones del usuario (`AddressResponseDTO`).

POST /addresses

autenticado

Crea una dirección. Cuerpo: `{ street, addressLine2?, city, province, postalCode, country }`.

PUT /addresses/{addressId}

autenticado

Actualiza una dirección.

DELETE /addresses/{addressId}

autenticado

Elimina una dirección (no permite borrar la predeterminada → `409`).

PATCH /addresses/{addressId}/default

autenticado

Marca una dirección como predeterminada.

8. Módulos del backend

Cada módulo sigue el mismo patrón de carpetas (`controller · service (+impl) · repository · dto · mapper · entity`). Puntos destacables:

- **product:** el listado usa *Specifications* (`ProductSpecification`) para componer filtros dinámicos (nombre, categoría, precio, marca) sobre una única consulta paginada. El estado del producto se recalcula según el stock.
- **category:** categorías autorreferenciadas (jerarquía). El `slug` se genera con `SlugUtils` ; la actualización valida que no se creen ciclos.
- **cart:** un carrito por usuario; añadir suma a la cantidad existente y valida stock; la respuesta incluye subtotales y total calculados.
- **wishlist:** favoritos con restricción única usuario+producto; el frontend mantiene un `Set` de ids para pintar el corazón igual en todas las vistas.
- **user:** la entidad implementa `UserDetails` ; separa el perfil propio (`/users/me`) de la administración (`/admin/users`).
- **config:** `DataInitializer` (admin) y `CatalogInitializer` (categorías + productos de demo), ambos idempotentes.

9. Frontend

SPA en React (Vite) que consume la API. Está organizado por capas dentro de `frontend/src/` :

Capa	Qué contiene
<code>api/</code>	<code>axiosClient</code> : instancia única de axios con interceptores que añaden el JWT en cada petición y gestionan el 401 (cierra sesión y redirige a login).
<code>services/</code>	Una función por operación de la API, agrupadas por dominio: <code>productService</code> , <code>cartService</code> , <code>categoryService</code> , <code>wishlistService</code> , <code>authService</code> .
<code>context/</code>	Estado global: <code>AuthContext</code> (sesión/usuario, persistida en <code>localStorage</code>) y <code>WishlistContext</code> (favoritos).
<code>hooks/</code>	Lógica reutilizable: <code>useAuth</code> , <code>useWishlist</code> , <code>useDebounce</code> (retrasa la búsqueda para no llamar en cada tecla).
<code>components/</code>	Piezas de UI: <code>ProductCard</code> , <code>PriceRangeSlider</code> , <code>Spinner</code> , <code>EmptyState</code> , iconos.
<code>pages/</code>	Vistas completas: <code>Home</code> (catálogo con búsqueda, filtros y paginación), <code>Cart</code> , <code>Favorites</code> , <code>ProductDetail</code> , <code>Login</code> , <code>Register</code> .
<code>layouts/</code> · <code>routes/</code>	<code>MainLayout</code> + <code>Sidebar</code> ; <code>AppRoutes</code> y <code>ProtectedRoute</code> (protege las vistas privadas comprobando la sesión).

Gestión de estado: se usan los hooks nativos de React. El estado (`useState`) que cambia provoca un re-render; los efectos (`useEffect`) cargan datos cuando cambian sus dependencias (filtros, sesión...). Valores como el total del carrito son *estado derivado*: se recalculan en cada render a partir del estado, no se almacenan.

10. Configuración

Centralizada en `src/main/resources/application.properties` :

Propiedad	Descripción
<code>spring.datasource.url / username / password</code>	Conexión a PostgreSQL. Debe coincidir con lo que expone <code>docker-compose.yml</code> .
<code>spring.jpa.hibernate.ddl-auto=update</code>	Hibernate crea/actualiza las tablas según las entidades, sin borrar datos. Válido en desarrollo; en producción se usaría <code>validate</code> + migraciones.
<code>spring.jpa.show-sql=true</code>	Muestra el SQL en consola (desarrollo).
<code>jwt.secret</code>	Clave secreta para firmar los tokens.
<code>jwt.expiration=3600000</code>	Duración del token en ms (1 hora).
<code>app.admin.email / password</code>	Credenciales del admin que crea el seeder al arrancar.
<code>stripe.* / spring.mail.*</code>	Configuración preparada para pagos (Stripe) y correo (SMTP), aún no en uso.

En un entorno real, los secretos (credenciales, `jwt.secret`) deben externalizarse a variables de entorno y no versionarse.

11. Puesta en marcha (onboarding)

Requisitos

- Java 21 · Docker Desktop · Node.js 18+ · (Maven va incluido con el wrapper `mvnw`).

Pasos

```
# 1) Clonar
git clone <url-del-repositorio>
cd smartcommerce

# 2) Base de datos (PostgreSQL en Docker)
docker compose up -d

# 3) Backend → http://localhost:8080
./mvnw spring-boot:run          # (Windows: .\mvnw.cmd spring-boot:run)

# 4) Frontend → http://localhost:5173
cd frontend
npm install                     # solo la primera vez
npm run dev
```

Verificación

- Backend vivo: `http://localhost:8080/swagger-ui.html` o `GET /api/v1/products`.
- Login de administrador (creado por el seeder): `admin@smartcommerce.com` / `Admin1234!`.

Cómo añadir un nuevo módulo (patrón)

1. Crea el paquete `com.smartcommerce.<módulo>` con las subcarpetas `entity`, `repository`, `dto/{request,response}`, `mapper`, `service (+impl)`, `controller`.
2. Define la entidad JPA y su repositorio (`extends JpaRepository`).
3. Crea los DTOs de entrada/salida y el mapper (`MapStruct`).
4. Implementa la lógica en el servicio; deja el controlador delgado.
5. Protege las rutas en `SecurityConfig` o con `@PreAuthorize` si aplica.

12. Datos de demostración (seeders)

Al arrancar el backend, dos inicializadores (`CommandLineRunner`) rellenan la base de datos de forma **idempotente** (solo crean lo que falta):

- `DataInitializer` : crea el usuario administrador si no existe.
- `CatalogInitializer` : crea 7 categorías y 18 productos de demostración (comprueba por `slug` y por `name` para no duplicar).

Gracias a esto, un clon nuevo del proyecto (o una base de datos levantada desde cero con `docker compose down -v`) arranca con catálogo listo, sin intervención manual.

13. Manejo de errores

Un `@RestControllerAdvice` (`GlobalExceptionHandler`) centraliza el manejo de excepciones y devuelve siempre la misma estructura (`ErrorResponse`): `{ timestamp, status, message }` .

Excepción	Código HTTP	Situación
<code>*NotFoundException</code> (Product, User, Category, Address, CartItem)	404 Not Found	El recurso solicitado no existe.
<code>EmailAlreadyExistsException</code>	409 Conflict	Registro con un email ya en uso.
<code>CannotDeleteDefaultAddressException</code>	409 Conflict	Intento de borrar la dirección predeterminada.
<code>BadCredentialsException</code>	401 Unauthorized	Login con credenciales incorrectas.
<code>MethodArgumentNotValidException</code>	400 Bad Request	Validación fallida (devuelve qué campos fallan).
<code>IllegalStateException</code>	400 Bad Request	Regla de negocio incumplida (p. ej. stock insuficiente, categoría inactiva).

14. Convenciones de desarrollo y flujo Git

Ramas

Modelo basado en `develop` / `main` con ramas de trabajo:

Rama	Uso
<code>main</code>	Versión estable / entregable.
<code>develop</code>	Integración del trabajo en curso.
<code>feature/<nombre></code>	Nueva funcionalidad.
<code>fix/<nombre></code>	Corrección de errores.
<code>docs/<nombre></code>	Documentación.

El flujo habitual: se trabaja en una rama `feature/` o `fix/`, se abre un **Pull Request** a `develop`, se revisa y se fusiona. Periódicamente, `develop` se promociona a `main`.

Commits

Se sigue **Conventional Commits**: `<tipo>(<ámbito>): <descripción>`.

```
feat(cart): cart page with quantity, selection and dynamic total
fix(cart): keep cart item order stable to prevent wrong-row edits
docs: add technical documentation
```

Tipos habituales: `feat` (funcionalidad), `fix` (corrección), `docs`, `refactor`, `chore`, `test`.

Estilo de código

- Backend: nunca exponer entidades (usar DTOs); controladores delgados; lógica en servicios; validación con anotaciones.
- Frontend: una función por llamada a la API en `services/`; estado global solo cuando lo necesitan varias vistas; componentes pequeños y con una responsabilidad.

15. Roadmap

Módulos y objetivos previstos (algunos ya con configuración base):

- **Pedidos (orders):** convertir el carrito en un pedido con su histórico.
- **Pagos (Stripe):** integración de pago (dependencia y configuración ya presentes).
- **Correo (SMTP):** notificaciones por email (configuración ya presente).
- **Contenerización completa:** Dockerizar backend y frontend para levantar toda la aplicación con un único `docker compose up`.

16. Glosario

Término	Definición
API REST	Interfaz basada en HTTP con URLs (endpoints) para operar sobre recursos.
DTO	Objeto de transferencia de datos; entrada/salida de la API, separado de la entidad.
Entidad	Clase Java mapeada a una tabla de la base de datos (JPA).
ORM (JPA/Hibernate)	Traduce entre objetos Java y filas de tablas.
JWT	Token firmado que autentica cada petición sin estado de sesión en el servidor.
Stateless	Sin sesión en servidor: cada petición se autentica de forma independiente.
CORS	Permiso que otorga el backend para ser llamado desde el origen del frontend.
BCrypt	Algoritmo de hash irreversible para almacenar contraseñas.
Specification	Pieza de filtro combinable para construir consultas dinámicas (Spring Data JPA).
SPA	Single-Page Application: cambia de vista sin recargar el navegador.
Seeder	Componente que rellena datos iniciales al arrancar, de forma idempotente.
Idempotente	Operación que se puede repetir sin cambiar el resultado (no duplica).

Documento vivo. Esta documentación debe evolucionar con el proyecto: al añadir un módulo o endpoint, actualiza las secciones 5 (modelo de datos), 7 (API) y 8 (módulos) para mantenerla fiel al código.